

REST API 설계부터 React UI까지 구현하는 Fullstack Developer



정다훈 Fullstack Developer · 총 5년 9개월

휴대폰	010-4346-0429	Email	dahun428@naver.com
주소	경기 구리시 인창동	병역	[군필] 육군 병장
Portfolio	github.com/dahun428-fx		

요약

Node.js, Java/Spring Boot, MySQL, REST API, React-TypeScript 기반으로 Web/Admin 서비스와 B2B 업무 시스템을 개발해 온 풀스택 개발자입니다. 화면 요구사항을 기준으로 API 응답 구조, 서버 로직, 데이터 모델, UI 상태 처리를 함께 설계하고 구현해 왔습니다.

대웅제약에서는 외주 중심으로 운영되던 검진 예약 서비스를 내부 개발 가능한 임직원 건강 플랫폼으로 전환했습니다. MySQL 데이터 모델, Spring Boot 서버 로직, REST API, Web/Admin 화면을 개발했고, Node.js API를 통해 여러 백엔드 응답을 React 화면에서 사용할 수 있는 구조로 정리했습니다. 9주 안에 108개 페이지·82개 화면 규모의 플랫폼 전환을 수행했고, 신규 건강관리 기능 3건을 DB 모델부터 화면까지 구현했습니다.

AI 챗봇 서비스에서는 Python으로 AI 챗봇 LLM 개발에 참여하고 React 렌더링 구조를 구현했습니다. LLM 응답에서 Markdown, 표, 링크, 차트, 오류 케이스가 화면에 안정적으로 전달되도록 응답 형식을 정규화했으며, React에서는 SSE 응답 수신, 중지 요청, 재시도, 오류 상태 처리, Markdown 렌더링을 구현했습니다. 3,493명 대상 PoC에서 AI 답변 출력 시간을 10초에서 4초로 줄였고, Playwright E2E와 Jenkins-GitLab CI/CD로 QA와 배포 검증을 자동화했습니다.

핵심 성과

1. Spring Boot·MySQL 기반 풀스택 기능 개발

검진 예약 서비스를 임직원 건강 플랫폼으로 전환하며 MySQL 데이터 모델, Spring Boot 서버 로직, REST API, Web/Admin 화면을 개발했습니다. 9주 내 108개 페이지·82개 화면 규모의 서비스를 전환했고, 신규 건강관리 기능 3건을 DB 모델부터 화면까지 구현했습니다.

2. Node.js API 구현과 화면 중심 응답 구조 정리

React 화면에서 필요한 데이터 구조를 기준으로 Node.js API를 구현했습니다. 여러 백엔드 응답을 화면별 요구사항에 맞게 정리하고, 필터링·정렬·역할별 조회 로직을 적용해 UI에서 바로 사용할 수 있는 형태로 구성했습니다.

3. 실시간 AI 응답 처리와 Markdown 렌더링 구현

LLM 응답을 SSE 기반으로 수신하고, 중지 요청·재시도·오류 상태 처리까지 화면 흐름에 반영했습니다. Python으로 AI 챗봇 LLM 개발에 참여하며 LLM 원문 응답을 화면 렌더링에 필요한 Markdown·구조화 데이터 형태로 정리하고, 표·목록·링크·차트·오류 케이스 처리 기준을 맞췄습니다. AI 답변 출력 시간은 10초에서 4초로 줄였고, 400건 발화 검증 기준 정상 출력률 100%를 확인했습니다.

4. 객체사별 UI 구조 분리와 반복 개발 시간 단축

객체사별 UI, 문구, 메뉴, 기능 노출 조건을 Config-Driven UI와 Base-Theme로 분리했습니다. 반복 화면은 공통 컴포넌트와 Custom Hook으로 정리해 개발·검증 시간을 90분에서 15분으로 줄였습니다.

5. Playwright E2E와 CI/CD 검증 자동화

Playwright 기반 주요 화면 E2E 600여 건을 자동화했습니다. Jenkins-GitLab CI/CD에 빌드, 테스트, 배포 전 검증 절차를 연결해 반복 QA 시간을 3시간에서 1시간으로 줄였고, 배포 리드타임을 10분에서 2분으로 단축했습니다.

총 5년 9개월

2025.07 ~ 재직중

대웅제약 AI추진팀

주요직무: Fullstack Developer / 백엔드·API 개발 / React 화면 개발 / AI 서비스 제품화

Spring Boot·MySQL 기반 서버 로직, REST API, Web/Admin 화면을 개발하며 AI 챗봇과 B2B 건강 플랫폼을 내부에서 개발·운영할 수 있는 구조로 전환했습니다. 화면 요구사항에서 출발해 Controller, Service, Query, DB, React UI까지 이어지는 기능 단위 개발을 수행했습니다.

Node.js API를 구현해 여러 백엔드 응답을 React 화면에서 사용할 수 있는 형태로 정리했습니다. AI 챗봇 LLM 개발에는 Python 기반으로 참여했고, React 화면에는 SSE 응답 처리, Markdown 렌더링, 오류 상태 처리를 적용해 렌더링 안정성을 높였습니다.

- MySQL 데이터 모델, Spring Boot 서버 로직, REST API, Web/Admin 화면 개발
- Node.js API 구현 및 화면별 응답 구조 정리
- Python 기반 AI 챗봇 LLM 개발 참여 및 화면 렌더링용 응답 구조 정리
- SSE 기반 LLM 응답 처리, 중지 요청, 재시도, 오류 상태 처리
- 3,493명 대상 AI 챗봇 PoC 고도화
- LLM 답변 출력 시간 10초 → 4초, 초기 서비스 진입 시간 30초 → 6초
- Lighthouse 91점, 발화 400건 검증 기준 정상 출력률 100% 확인
- Playwright 600여 건 E2E 자동화, Jenkins CI/CD 구축, FE AX SOP 문서화

2020.10 ~ 2025.06

내담씨앤씨 SI사업부 대리

주요직무: 웹개발자 / 프론트엔드 개발자 / REST API 연동

B2B 커머스, 생산관리 대시보드, React Native 모바일 앱 프로젝트에서 화면 개발과 Spring REST API 연동을 수행했습니다. 필터링·정렬 로직, 역할별 데이터 조회 구조, 대용량 데이터 렌더링, 모바일 앱 API 호출 구조를 다뤘습니다.

한국미스미 프로젝트에서는 PHP·jQuery 기반 글로벌 B2B 커머스를 React·Next.js·TypeScript 구조로 전환해 페이지 접근 시간을 8초에서 2초로 줄였습니다. 삼성물산 프로젝트에서는 Vue 3·ECharts·RealGrid 기반 대시보드와 React Native 앱의 데이터 연동, 성능, 사용성을 개선했습니다.

- PHP/jQuery 기반 B2B 쇼핑몰을 React/Next.js/TypeScript 구조로 단계적 전환
- 상품 비교·멀티 다운로드 기능을 공통 컴포넌트와 Custom Hook으로 정리
- Vue 3·ECharts·RealGrid 기반 대용량 생산 공정 데이터 시각화 대시보드 개발
- Spring REST API, 필터링·정렬 로직, 역할별 데이터 조회·표현 구조 구현
- React Native iOS/Android 앱 기능 개선 및 배포 대응
- Vercel 배포 자동화, Datadog·GA·Adobe Analytics 기반 모니터링 운영

Backend · API

TypeScript · Node.js · REST API · Java · Spring Boot · MySQL · MyBatis · Oracle · Python

- Spring Boot·MySQL 기반 데이터 모델, 서버 로직, REST API 구현
- Node.js API 구현 및 화면별 응답 구조 정리
- 필터링·정렬, 역할별 조회 로직 구현
- API 요청부터 Service 로직, Query, DB 조회, 화면 반영까지 기능 단위 개발
- Python 기반 AI 챗봇 LLM 개발 참여 및 화면 렌더링용 응답 구조 정리

Frontend

React · Next.js · Vue 3 · React Native · Redux · Recoil · TanStack Query · Tailwind CSS

- React·Vue 3 기반 SPA 화면 개발 및 운영
- Next.js 기반 SSR·CSR 렌더링 전략 분리
- React Native 기반 iOS·Android 앱 기능 개발 및 배포
- 반복 UI를 공통 컴포넌트와 Custom Hook으로 정리

Architecture · UI Structure

Config-Driven UI · Base-Theme · 공통 컴포넌트 · Custom Hook · Storybook

- 고객사별 UI, 문구, 메뉴, 기능 노출 조건을 Config-Driven UI와 Base-Theme로 분리
- 반복 기능을 공통 컴포넌트와 Custom Hook으로 정리
- Storybook 기반 컴포넌트 상태별 검증

Realtime · AI · Visualization

SSE · Markdown Renderer · ECharts · RealGrid · Chart.js · Firebase FCM

- SSE 기반 LLM 응답 처리, 중지 요청, 재시도, 오류 상태 처리
- Markdown, 표, 목록, 링크, 차트를 React 컴포넌트로 렌더링
- 대용량 테이블·시계열 데이터에 Lazy Rendering 적용
- Firebase FCM 기반 모바일 알림 연동

Quality · Build · Delivery

Playwright · Jest · Storybook · ESLint · Jenkins · GitLab CI/CD · Vercel · GA4 · PostHog · Datadog

- Playwright 기반 주요 화면 E2E 자동화
- Jenkins·GitLab CI/CD 기반 빌드·테스트·배포 검증
- Vercel 자동 배포
- GA4·PostHog·Datadog 기반 운영 지표 확인
- AI 생성 코드 확인 기준(FE AX SOP) 문서화

Languages · Foundations

TypeScript · JavaScript · Java · Python · PHP

- TypeScript 기반 React 화면 및 API 연동 개발
- Java/Spring Boot 기반 서버 로직 및 REST API 구현
- Python 기반 AI 챗봇 LLM 개발 참여 및 화면 렌더링용 응답 구조 정리
- PHP·jQuery 레거시 코드 분석 및 React/Next.js 전환

학력

2010.03 ~ 2017.02 **성공회대학교 일어일본학과 졸업**

복수전공: 경영학과

학점: 3.8 / 4.5

2010 **여의도고등학교 졸업**

교육

2020.03 ~ 2020.09 웹 콘텐츠 개발을 위한 응용SW엔지니어링

중앙에이치티에이(주)

Java 기반 웹 애플리케이션의 백엔드와 프론트엔드 개발 과정을 이수하고, Spring Framework 기반 팀 프로젝트를 진행했습니다.

습득 기술:

HTML5, JavaScript, jQuery, Vue.js, CSS, JSP, Java, Spring Framework, AWS, RESTful API, OracleDB, MariaDB, Git/GitHub, Subversion, Tomcat, Eclipse

자격증

정보처리기사 한국산업인력공단

해외경험

2018.09 ~ 2019.06 일본 워킹홀리데이 10개월

무인양품 일본지사 도쿄 근무

어학

영어

취득일: 2024.05

TOEIC 825점

일본어

취득일: 2018.08

JLPT N1

《 대응계약 | B2B 플랫폼 풀스택 내재화 》

외주 운영 검진 예약 서비스를 내부 개발 가능한 B2B 건강 플랫폼으로 전환

2025.11 ~ 2026.02

책임 범위 서비스·데이터 구조 분석, MySQL·Spring Boot·REST API·Web/Admin 개발, Node.js API 구현, 권한·메뉴 정책 공통화, WebView 앱 운영, CI/CD

[문제]

검진 예약 서비스는 외주 중심으로 운영되어 내부에서 기능 변경과 배포를 빠르게 처리하기 어려웠습니다. 기존 jQuery·Thymeleaf 화면과 Spring Boot·MySQL 구조가 기능별로 결합되어 변경 영향 범위 파악이 어려웠고, 고객사별 CI, 메뉴, 기능 노출 정책도 일관되게 관리되지 않았습니다.

[주요 실행]

- 화면, API, Service 로직, Query, DB 조회 구조를 확인해 기능별 변경 영향 범위 정리
- 신규 건강관리 기능의 MySQL 데이터 모델, Spring Boot 서버 로직, REST API, Web/Admin 화면 개발
- jQuery·Thymeleaf와 React가 화면 단위로 공존하는 점진적 전환 구조 적용
- 사용자 권한, 메뉴, 브랜딩, 데이터 출력 정책을 공통 기준으로 정리
- Node.js API를 구현해 여러 백엔드 응답을 React 화면에서 사용할 수 있는 형태로 구성
- Jenkins 기반 빌드·검증·배포 파이프라인 구성
- WebView 환경에서의 동작 검증 기준 정리

[성과]

- 9주 내 108개 페이지·82개 화면 규모의 임직원 건강 플랫폼으로 전환
- 건강 데이터를 차트·대시보드로 보여주는 신규 기능 3건을 DB 모델부터 화면까지 개발
- 외주 의존 기능 변경·배포를 DB, 서버, Web/Admin, App 영역에서 내부 대응 가능하도록 개선
- 고객사별 CI, 메뉴, 기능 노출 변경을 1주 내 대응 가능한 구조로 정리

기술 React, TypeScript, jQuery, Thymeleaf, Java, Spring Boot, Node.js, REST API, MySQL, Tailwind CSS, Jenkins CI/CD, WebView, iOS, Android

《 대응계약 | AI 실시간 서비스 도입·공통 화면 구조 정리 》

SSE 기반 LLM 응답 처리와 고객사별 AI 챗봇 화면 개발

2025.07 ~ 2025.10

책임 범위 React 화면 개발, AI·백엔드 응답 정책 조율, Python 기반 AI 챗봇 LLM 개발 참여, SSE 응답 처리, Markdown 렌더링, 공통 컴포넌트 구조 정리

[문제] 초기 AI 건강검진 챗봇은 답변 생성이 완료된 뒤 한 번에 출력되는 구조였습니다. 사용자는 답변이 나올 때까지 기다려야 했고, Markdown, 표, 링크, 차트 등 비정형 응답을 안정적으로 렌더링하는 구조도 부족했습니다. 고객사별 화면, 테마, 기능 노출 조건도 일관되게 관리하기 어려웠습니다.

[주요 실행]

- AI 개발자와 API 응답 형식, SSE 스트리밍 방식, 오류 처리 정책 조율
- SSE 기반 LLM 응답 수신, 중지 요청, 재시도, 오류 상태 처리 구현
- Markdown 전용 렌더링 계층 구현
- 표, 목록, 링크, 차트를 React 컴포넌트로 변환
- Python AI 챗봇의 프롬프트 구성, 응답 후처리 로직, 화면 렌더링용 데이터 구조 개선
- XSS 필터링, 404·500·LLM 오류 상태 처리 적용
- 공통 화면, 테마, 컴포넌트를 Base-Theme로 분리
- 고객사별 화면 차이를 Config-Driven UI로 관리
- React Query 캐싱, 코드 스플리팅, 이미지 최적화, HTTP/2 전환 적용

[성과]

- 3,493명 규모 AI 챗봇 PoC 운영
- 만족도 조사 기준 사용성 4.18, 서비스 만족도 4.06, 완성도 4.05
- AI 답변 출력 시간 10초 → 4초
- 초기 서비스 진입 시간 30초 → 6초
- Lighthouse 91점
- 400건 발화 검증 기준 답변 화면 정상 출력을 100% 확인
- 공통 화면·테마·컴포넌트를 재사용 가능하게 분리해 신규 AI 챗봇 구축 기간을 기존 10주 수준에서 약 2주 수준으로 줄일 수 있는 구조 마련

기술 React, TypeScript, TanStack Query, Recoil, SSE, Python, Chart.js, Tailwind CSS

《 대응계약 | 테스트·빌드·배포 자동화 및 AI 활용 개발 프로세스 》

QA·배포 검증 자동화와 AI 생성 코드 확인 기준 정리

2026년 상반기

적용 서비스 AI코치, 비즈36.5, 바이오에이지

책임 범위 빌드·배포 파이프라인 자동화, 테스트·배포 전후 검증 절차 구성, 운영 지표 대시보드 구성, FE 개발 기준 문서화

[문제]

서비스 수와 공통 컴포넌트가 늘어나면서 수동 QA, 반복 UI 개발, 배포 전후 확인, 운영 지표 조회가 병목이 되었습니다. AI 생성 코드의 품질, 보안, 일관성을 확인할 기준도 필요했습니다.

[주요 실행]

- Playwright 기반 주요 사용자 흐름 E2E 회귀 시나리오 작성
- Storybook 기반 컴포넌트 상태별 확인 절차 구성
- LLM 응답 확인, 타입 체크, 테스트, 빌드, 배포 전 확인 항목을 검증 절차에 반영
- Jenkins-GitLab CI/CD 기반 빌드·테스트·배포 검증 자동화
- GA4-PostHog 기반 운영 지표 대시보드 구성
- AI 생성 코드의 컨텍스트, 금지 패턴, 보안 위험, 검증 절차를 FE AX SOP로 문서화
- 개발 세미나 12회 운영
- 기술 문서 48건 작성

[성과]

- Playwright 기반 E2E 회귀 시나리오 600여 건 자동화
- 반복 QA 시간 3시간 → 1시간
- 반복 컴포넌트 개발·검증 시간 90분 → 15분
- 운영 데이터 확인 절차 3단계 → 1단계
- 기획-개발 검증 리드타임 2일 → 1일
- 테스트 커버리지 98% 수준 확보
※ 내부 측정 기준

기술 Playwright, Storybook, Jest, ESLint, TypeScript, Jenkins, GitLab CI/CD, GA4, PostHog

« 삼성물산 | 데이터 플랫폼 · 모바일 애플리케이션 »

Vue 3 대용량 데이터 대시보드 성능 개선 및 React Native 모바일 앱 개발

2024.10 ~ 2025.04

책임 범위 대용량 데이터 시각화 성능 개선, Spring REST API 연동, React Native iOS·Android 개발·배포

[문제] Web 대시보드는 대용량 테이블과 시계열 데이터를 빠르게 표시해야 했습니다. 모바일 앱은 검색 시 반복 API 호출이 발생했고, 플랫폼별 동작 차이로 사용자 대기 시간과 운영 부담이 있었습니다.

[주요 실행]

- Vue 3-ECharts·RealGrid 기반 대용량 데이터 시각화 화면 개발
- 테이블·차트 렌더링 생명주기 분리
- Lazy Rendering 적용
- Spring REST API, 필터링·정렬 로직 구현
- 사용자 역할별 데이터 조회·표현 구조 구성
- 검색 시마다 API를 호출하던 구조를 화면 진입 시 비동기로 미리 조회한 뒤 Recoil에 저장하는 방식으로 변경
- React Native 기반 iOS·Android 공통 기능 개발
- Firebase FCM 실시간 알림 연동
- 플랫폼별 배포·운영 대응

[성과]

- 대시보드 렌더링 시간 2.5초 → 1초대
- React Native 앱 검색 응답 시간 5초 → 1초
- Web 데이터 연동, 대시보드 렌더링 최적화, iOS·Android 기능 개발·배포 수행

기술 Vue 3, React Native, TypeScript, ECharts, RealGrid, Java, Spring, REST API, MyBatis, MySQL, Recoil, SQLite, Firebase FCM

« 한국미스미 | 글로벌 B2B 커머스 »

PHP·jQuery 레거시의 React·Next.js 전환 및 성능 개선

2022.06 ~ 2024.10

책임 범위 PHP·jQuery 레거시 분석, Next.js 전환, 렌더링 전략 분리, 성능 개선, SEO, 다국어, 배포 자동화, 모니터링

[문제] PHP·jQuery 기반 글로벌 쇼핑몰은 화면 결합도가 높아 확장과 유지보수가 어려웠습니다. 초기 접근 시간이 길었고, 모든 페이지에 같은 렌더링 방식을 적용해 성능과 검색 노출을 함께 최적화하기 어려웠습니다. 다국어 환경과 국가별 배포 요건도 함께 고려해야 했습니다.

[주요 실행]

- PHP·jQuery 레거시 화면을 Next.js·React·TypeScript 컴포넌트 구조로 단계적 전환
- 페이지 특성에 따라 SSR·CSR 렌더링 방식 분리
- 상품 비교·멀티 다운로드 기능을 공통 컴포넌트와 Custom Hook으로 정리
- Redux 기반 상태 관리 적용
- Lighthouse 병목 분석
- Lazy Loading, 비동기 API 호출, 리소스 최적화 적용
- i18next 기반 다국어 구조 적용
- Vercel 배포 자동화
- Datadog, GA, Adobe Analytics 기반 운영 모니터링 구성

[성과]

- 페이지 접근 시간 8초 → 2초
- Next.js 전환 후 기존 PHP 서비스 대비 평균 로딩 속도 약 50% 개선
- 화면 개발, 렌더링 전략, 성능 개선, 다국어, 배포 자동화, 모니터링 수행

기술 Next.js, React, TypeScript, JavaScript, Redux, RxJS, i18next, PHP, jQuery, Twig, Vercel, Datadog, Lighthouse, GA, Adobe Analytics